Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Simulador de Administración de Memoria Virtual

con Despacho de Procesos, Arquitectura Cliente-Servidor y Balanceo de Carga mediante Thread Pool

Curso: Sistemas Operativos II

Profesora: Dra. Hilda Castillo Zacatelco

Semestre: Primavera 2026

Nombre
Razo Montañez Gael Askary
Ángel Hernández González
José Gabriel Hernández Mejía

Mayo 2026

Índice

1. Introducción **5**

2. Marco Teórico **6**
2.1. Memoria Virtual y Paginación . . . 6
2.2. Fallos de Página . . . 6
2.3. Algoritmos de Reemplazo . . . 7
2.3.1. FIFO . . . 7
2.3.2. LRU (Least Recently Used) . . . 7
2.3.3. Clock (Segunda Oportunidad) . . . 8
2.3.4. ARC (Adaptive Replacement Cache) . . . 8
2.3.5. CAR (Clock with Adaptive Replacement) . . . 8
2.3.6. Comparación de Algoritmos . . . 8
2.4. Despacho — Round Robin . . . 8
2.5. Programación Concurrente — Thread Pool . . . 9
2.6. Arquitectura Cliente-Servidor TCP . . . 9

3. Diseño del Sistema **10**
3.1. Visión General de la Arquitectura . . . 10
3.2. Diagrama de Clases Completo . . . 11
3.3. HybridPolicy — Diseño Detallado . . . 11
3.4. MemoryManager — Modelo de Dos Niveles . . . 13
3.5. Dispatcher — Round Robin con String de Referencias . . . 13
3.6. Thread Pool — Diseño Detallado . . . 14
3.7. Protocolo de Red . . . 14
3.8. Interfaz Gráfica (Qt6) . . . 15

4. Implementación **16**
4.1. ThreadPool . . . 16
4.2. HybridPolicy — Métodos Críticos . . . 17
4.3. MemoryManager — accessProcessPage . . . 18
4.4. Dispatcher — step() y rotateProcess() . . . 19
4.5. Server — Accept Loop y handleClient . . . 20
4.6. Sincronización — Análisis de Condiciones de Carrera . . . 21

5. Manual Técnico **23**
5.1. Estructura del Repositorio . . . 23
5.2. Dependencias y Versiones . . . 24
5.3. API de la Biblioteca Central . . . 24
5.3.1. Clase MemoryManager . . . 24
5.3.2. Clase Dispatcher . . . 24
5.3.3. Clase ThreadPool . . . 25
5.4. Protocolo TCP — Especificación Formal . . . 25
5.5. Extensibilidad — Añadir un Nuevo Algoritmo . . . 26

5.6. Configuración del Thread Pool 26

6. Manual de Usuario **28**
6.1. Requisitos del Sistema 28
6.2. Instalación y Compilación 28
6.3. Guía de la Interfaz Gráfica 28
6.3.1. Pantalla Principal 29
6.3.2. Configuración Inicial 29
6.3.3. Crear un Proceso 30
6.3.4. Ejecutar Paso a Paso 31
6.3.5. Alerta de Fallo de Página 32
6.3.6. Tabla de Páginas del Proceso 33
6.3.7. Reporte Final 34
6.4. Guía del Servidor y Cliente (Parte 2) 34
6.4.1. Iniciar el Servidor 34
6.4.2. Enviar un Proceso desde un Cliente 35
6.4.3. Monitorear el Servidor 35
6.5. Interpretación de Resultados 35
6.6. Solución de Problemas 36

7. Pruebas y Resultados **37**
7.1. Metodología de Pruebas 37
7.2. Prueba 1 — Alta Conflictividad (Thrashing) 37
7.3. Prueba 2 — Localidad de Referencia 38
7.4. Prueba 3 — Concurrencia (Parte 2) 38
7.5. Análisis Comparativo Adicional 39

8. Discusión **40**

9. Conclusiones **41**

10. Referencias **42**

Índice de figuras

1. Proceso de traducción de direcciones lógicas a físicas mediante paginación. 6
2. Diagrama de flujo del manejador de fallos de página. 7
3. Diagrama de Gantt mostrando un despacho Round Robin simplificado. . . 9
4. Diagrama de secuencia de la interacción cliente-servidor mediante sockets TCP. 9
5. Arquitectura de dos capas del simulador MemManager. 10
6. Diagrama de clases UML del núcleo del simulador. 11
7. Máquina de estados de la política de reemplazo híbrida CAR. 12
8. Mapeo de memoria virtual a memoria física mediante la tabla global de marcos. 13
9. Arquitectura multihilo con ThreadPool. 14
10. Mockup estructural de la ventana principal de la interfaz. 15
11. Curva teórica de rendimiento máximo esperado según la Ley de Amdahl. . 27
12. Vista general de la aplicación MemManager GUI. 29
13. Panel de configuración del simulador. 30
14. Creación de un proceso interactivo. 31
15. Ejecución interactiva y bitácora. 32
16. Alerta interactiva del simulador. 32
17. Visor de la Tabla de Páginas. 33
18. Reporte de terminación. 34
19. Fallos de página por proceso (Prueba 1). 37
20. Comparación de fallos de página entre escenarios. 38
21. Comparación de Tiempos de Espera. 39

Índice de cuadros

1. Comparación de algoritmos de reemplazo de páginas. 8
2. Transiciones de estado del Dispatcher. 13
3. Especificación de los mensajes del protocolo TCP. 14
5. Solución de problemas comunes. 36
6. Resultados bajo condiciones de Thrashing. 37
7. Resultados con alta localidad de referencia. 38
8. Métricas del monitor durante un ataque en ráfaga (burst). 38

1 Introducción

El diseño de los sistemas operativos modernos está intrínsecamente ligado al manejo eficiente de la jerarquía de memoria. En sistemas multiprogramados, la memoria principal (RAM) es un recurso limitado y compartido que debe ser protegido y administrado cuidadosamente para asegurar la estabilidad del sistema. La memoria virtual surge como una solución elegante a este problema, proveyendo a cada proceso la ilusión de poseer un espacio de direcciones grande y contiguo, aislado del resto de los procesos y de la ubicación física real de los datos. La paginación es el mecanismo dominante de memoria virtual en las arquitecturas contemporáneas, como x86-64. Al dividir la memoria en bloques de tamaño fijo (páginas lógicas y marcos físicos), el sistema operativo simplifica la asignación y elimina la fragmentación externa. Sin embargo, dado que el espacio de direcciones virtuales supera con creces la capacidad de la memoria física, es inevitable que ocurran fallos de página. Un fallo de página obliga al sistema a suspender el proceso, acceder al disco (un dispositivo de almacenamiento secundario significativamente más lento) y cargar la página requerida en un marco físico. Cuando la RAM está llena, el sistema operativo debe decidir qué marco liberar para dar cabida a la nueva página. Esta decisión es gobernada por el algoritmo de reemplazo de páginas. Una elección deficiente puede llevar a la expulsión de una página que será requerida inmediatamente, provocando una avalancha de fallos de página que degrada severamente el rendimiento del sistema, un fenómeno conocido como *thrashing*. La teoría del conjunto de trabajo (working set) establece que los algoritmos de reemplazo deben retener en memoria aquellas páginas que el proceso necesita activamente, adaptándose a los cambios de localidad de referencia a lo largo del tiempo. El objetivo principal de este proyecto es desarrollar un simulador pedagógico paso a paso que exponga de manera transparente todo el estado interno de la administración de memoria virtual. A diferencia de un kernel real, donde estas estructuras están ocultas, este simulador permite visualizar el estado de la tabla de páginas, la matriz de memoria física y las listas de la política de reemplazo en tiempo real. Además, para dotar al proyecto de relevancia industrial y conectarlo con los paradigmas actuales de concurrencia y redes, el simulador ha sido extendido con una arquitectura cliente-servidor basada en sockets TCP. En lugar de un modelo ingenuo de un hilo por cliente, se ha implementado un mecanismo de balanceo de carga mediante un *Thread Pool*. Esta arquitectura no solo optimiza el uso de recursos del sistema anfitrión, sino que simula un entorno de procesamiento por lotes concurrente donde múltiples procesos de diferentes usuarios compiten por los recursos de la CPU y la memoria. En los siguientes capítulos, este documento desglosará el marco teórico subyacente (Capítulo 2), la arquitectura y diseño del sistema (Capítulo 3), los detalles críticos de la implementación en C++20 (Capítulo 4), el manual técnico para desarrolladores (Capítulo 5), la guía de uso de la interfaz gráfica y de línea de comandos (Capítulo 6), los resultados de las pruebas de estrés (Capítulo 7), y finalmente una discusión y conclusión sobre los aprendizajes obtenidos.

2 Marco Teórico

2.1 Memoria Virtual y Paginación

La memoria virtual abstrae la memoria principal de un dispositivo de almacenamiento masivo, típicamente un disco duro o SSD. La unidad fundamental de este esquema es la *página*, un bloque contiguo de memoria, usualmente de 4 KB. Las direcciones generadas por la CPU son direcciones lógicas (o virtuales), las cuales deben ser traducidas a direcciones físicas por una unidad de hardware llamada MMU (Memory Management Unit). La traducción se realiza mediante una estructura de datos mantenida por el sistema operativo denominada *Tabla de Páginas*. Para acelerar este proceso, la MMU cuenta con una caché de traducciones conocida como TLB (Translation Lookaside Buffer). Cada entrada de la tabla de páginas (PTE) no solo contiene el número de marco físico correspondiente, sino también bits de control como el bit de validez (indica si la página está en RAM), el bit de modificación o sucio (indica si la página ha sido escrita y requiere guardarse en disco) y los bits de protección (lectura, escritura, ejecución).

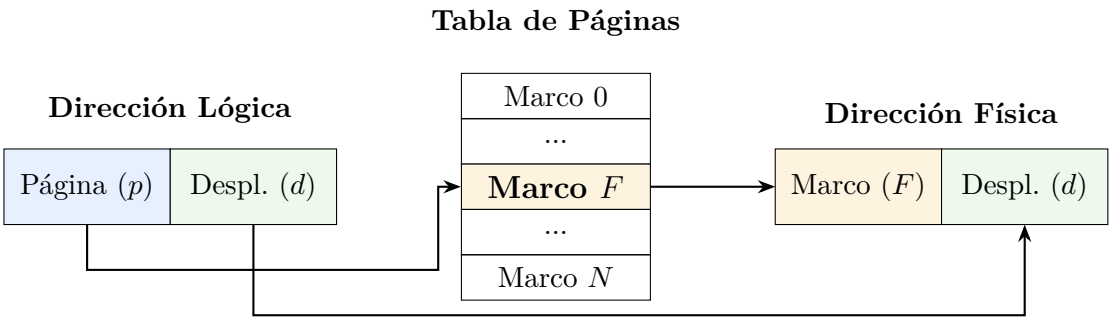

Figura 1: Proceso de traducción de direcciones lógicas a físicas mediante paginación.

2.2 Fallos de Página

Un fallo de página ocurre cuando la MMU intenta traducir una dirección lógica cuya entrada en la tabla de páginas tiene el bit de validez en cero, indicando que la página reside en disco. Esto genera un *trap* al sistema operativo.

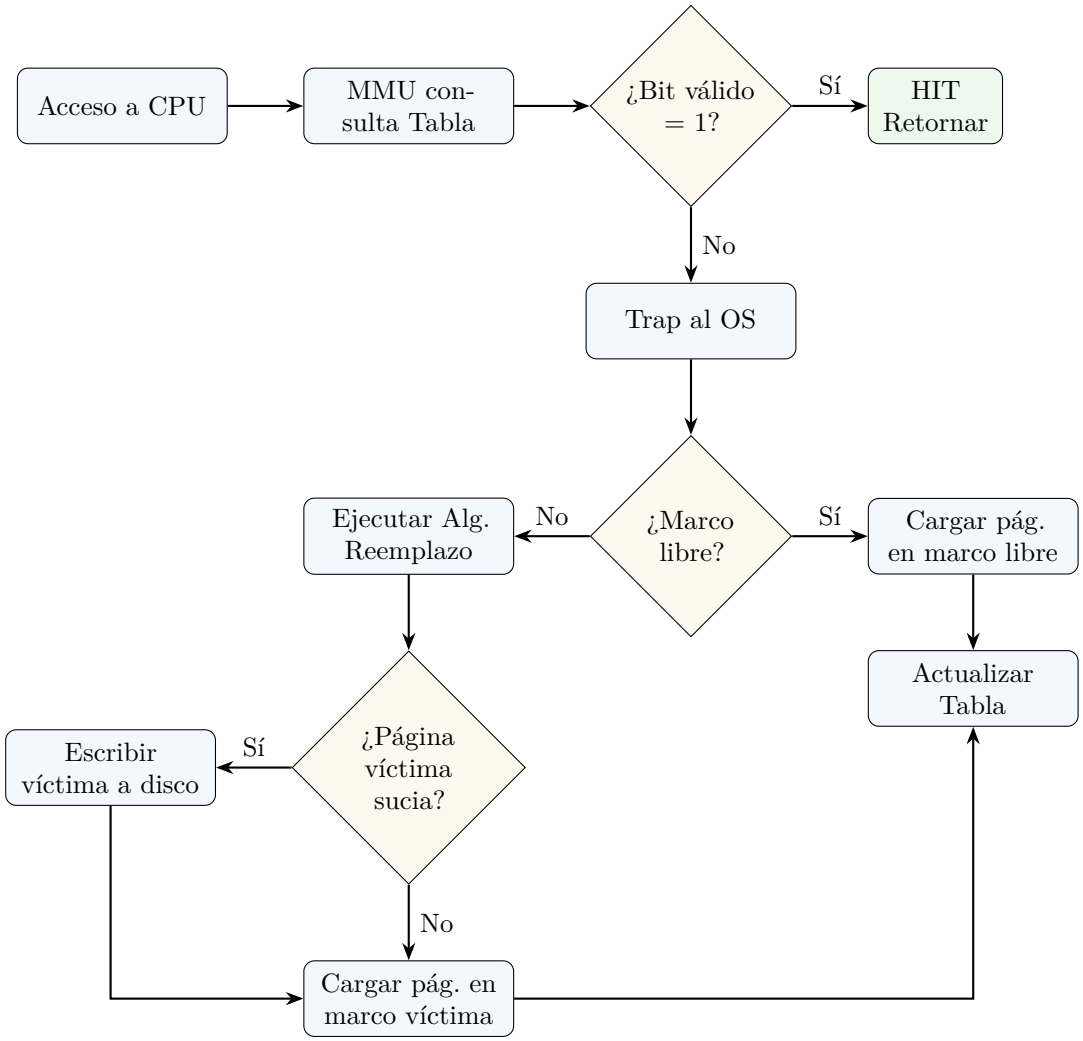

Figura 2: Diagrama de flujo del manejador de fallos de página.

2.3 Algoritmos de Reemplazo

2.3.1 FIFO

El reemplazo FIFO (First-In, First-Out) expulsa la página que lleva más tiempo en memoria. Aunque es simple, sufre de la anomalía de Bélády: aumentar el número de marcos de RAM puede *aumentar* el número de fallos de página.

2.3.2 LRU (Least Recently Used)

LRU expulsa la página que no ha sido accedida por más tiempo. Es libre de anomalías y aproxima bien el principio de localidad. Sin embargo, su implementación exacta (usando pilas o marcas de tiempo en cada acceso) requiere soporte extenso de hardware y es costosa.

2.3.3 Clock (Segunda Oportunidad)

Aproxima LRU usando un bit de referencia en la tabla de páginas. Una manecilla recorre los marcos; si el bit es 1, le da una "segunda oportunidad" (lo pone en 0) y avanza. Si es 0, lo reemplaza. Es eficiente y fácil de implementar.

2.3.4 ARC (Adaptive Replacement Cache)

ARC mantiene dos listas principales: T1 (recientemente accedidas, localidad de recencia) y T2 (frecuentemente accedidas, localidad de frecuencia), junto con historiales fantasma B1 y B2 para las páginas expulsadas. ARC adapta dinámicamente un parámetro p (tamaño objetivo de T1) según los hits en B1 o B2.

2.3.5 CAR (Clock with Adaptive Replacement)

CAR es el algoritmo implementado en la política híbrida del simulador. Combina la adaptabilidad de ARC con la eficiencia de Clock. En lugar de mantener listas LRU exactas para T1 y T2, CAR utiliza manecillas de reloj sobre listas circulares, obteniendo un costo de selección de víctima amortizado de $\mathcal{O}(1)$, manteniendo la resistencia al thrashing y la adaptación entre frecuencia y recencia.

2.3.6 Comparación de Algoritmos

Algoritmo	Complejidad	Sin Belády	Frecuencia	Adaptativo	Costo Impl.
FIFO	$\mathcal{O}(1)$	No	No	No	Bajo
LRU	$\mathcal{O}(1)$ (hardware)	Sí	No	No	Alto
Clock	$\mathcal{O}(1)$ amortizado	No	No	No	Bajo
ARC	$\mathcal{O}(1)$ amortizado	Sí	Sí	Sí	Medio-Alto
CAR	$\mathcal{O}(1)$ amortizado	Sí	Sí	Sí	Medio

Cuadro 1: Comparación de algoritmos de reemplazo de páginas.

2.4 Despacho — Round Robin

Round Robin asigna la CPU a cada proceso por un tiempo fijo llamado quantum (q). Las fórmulas fundamentales son:

- Tiempo de retorno (Turnaround): $T_r = T_f - T_a$ (Tiempo final - Tiempo de llegada)
- Tiempo de espera: $T_w = T_r - T_{burst}$

Ejemplo: 3 procesos con ráfagas y un $q = 5$.

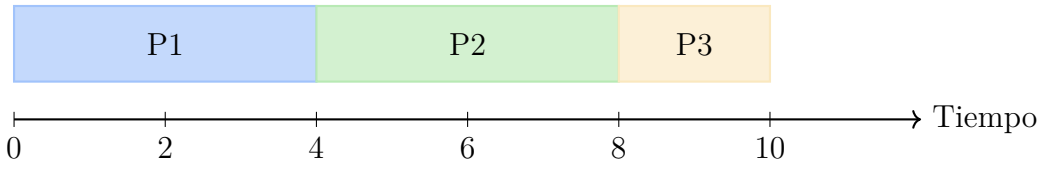

Figura 3: Diagrama de Gantt mostrando un despacho Round Robin simplificado.

2.5 Programación Concurrente — Thread Pool

El patrón *Thread Pool* mantiene un conjunto fijo de hilos trabajadores que consumen tareas de una cola compartida. Se protege la cola mediante un `std::mutex` y se señaliza la disponibilidad de tareas mediante `std::condition_variable`. Este modelo previene la explosión de hilos (thread explosion) y el sobrecosto de cambios de contexto asociado a crear un hilo por cada conexión TCP entrante, permitiendo escalar a miles de conexiones usando un número constante de hilos igual al número de núcleos físicos de la CPU. Para evitar la inversión de prioridad, hilos de mantenimiento (como el *stepper* de simulación) pueden usar `std::try_to_lock`.

2.6 Arquitectura Cliente-Servidor TCP

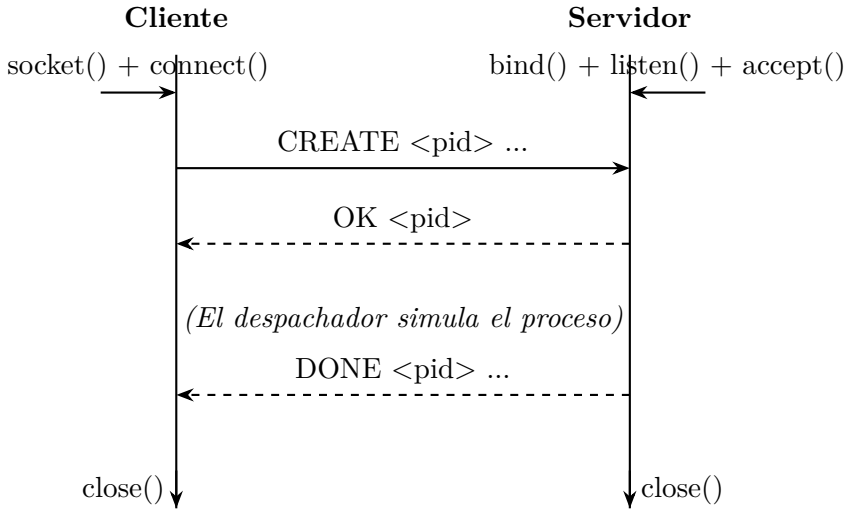

Figura 4: Diagrama de secuencia de la interacción cliente-servidor mediante sockets TCP.

3 Diseño del Sistema

3.1 Visión General de la Arquitectura

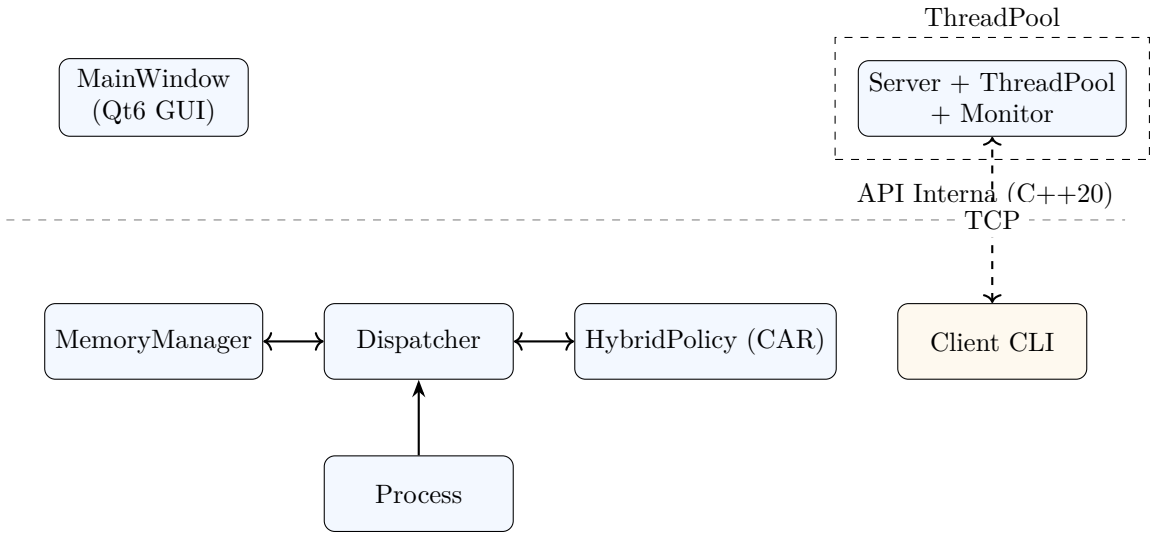

Figura 5: Arquitectura de dos capas del simulador MemManager.

3.2 Diagrama de Clases Completo

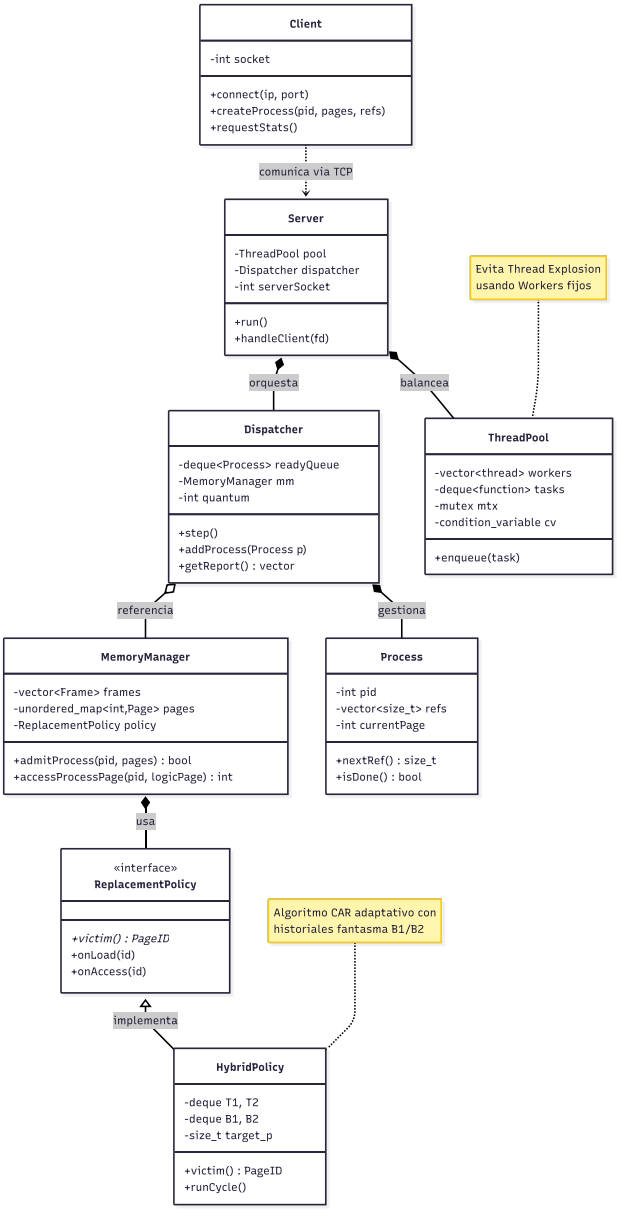

Figura 6: Diagrama de clases UML del núcleo del simulador.

3.3 HybridPolicy — Diseño Detallado

La clase `HybridPolicy` implementa el algoritmo CAR, utilizando cuatro estructuras de datos:

- **T1**: Una `std::deque` de entradas (ID de página y bit de referencia R). Mantiene páginas recientemente cargadas (vistas una vez).

- **T2**: Una `std::deque` de entradas. Mantiene páginas frecuentemente accedidas (vistas múltiples veces).
- **B1**: Un historial fantasma (solo IDs) de páginas expulsadas de T1.
- **B2**: Un historial fantasma de páginas expulsadas de T2.
- **p_**: El tamaño objetivo de T1. Se adapta dinámicamente: aumenta si hay un hit en B1, disminuye si hay un hit en B2.

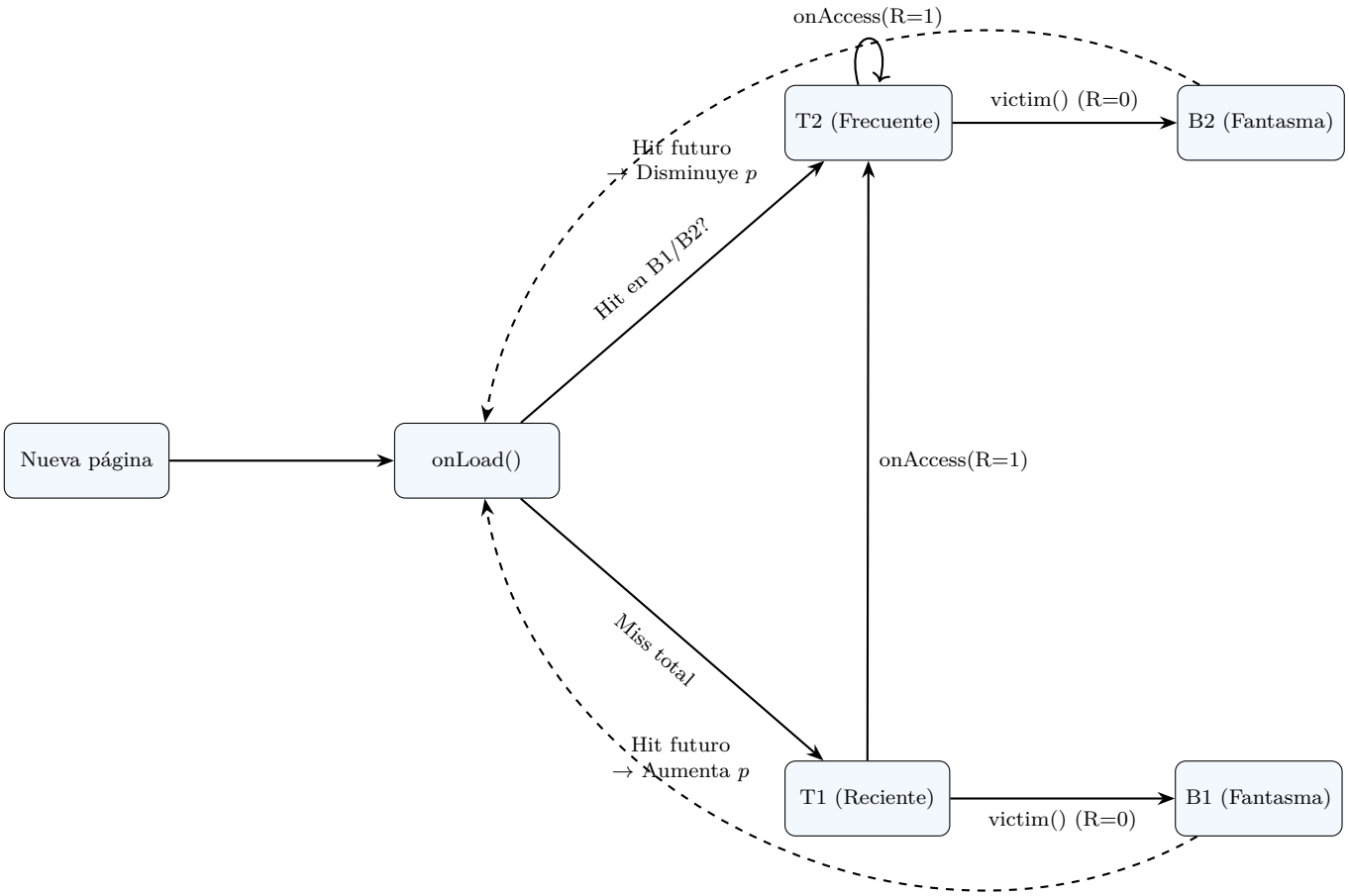

Figura 7: Máquina de estados de la política de reemplazo híbrida CAR.

Para simular el barrido de la manecilla de reloj (Clock) sin necesidad de recorrer explícitamente listas enlazadas en cada acceso, la `HybridPolicy` incorpora un método `runCycle()`. Cada `REFS_PER_CYCLE` (por defecto 3 referencias de memoria gestionadas por el `MemoryManager`), se ejecuta un "tick" del reloj que limpia todos los bits R (los pone en `false`) en T1 y T2. Esta aproximación mantiene el costo temporal por referencia

estrictamente acotado, aliviando el simulador de iteraciones costosas sobre colecciones largas.

3.4 MemoryManager — Modelo de Dos Niveles

El `MemoryManager` gestiona el espacio de direcciones con dos mapas de bits principales: `bitmap_` (páginas virtuales asignadas) y `inRam_` (páginas residentes en memoria física).

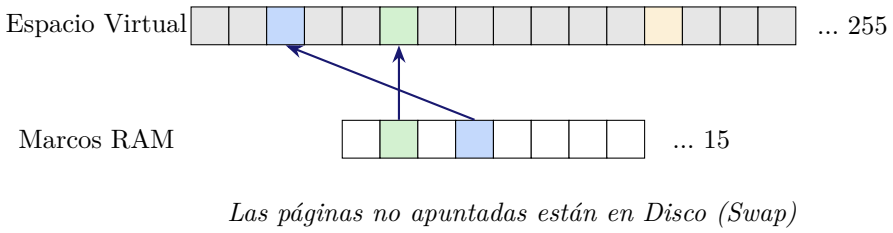

Figura 8: Mapeo de memoria virtual a memoria física mediante la tabla global de marcos.

3.5 Dispatcher — Round Robin con String de Referencias

El despachador es responsable de ejecutar el algoritmo Round Robin simulado. En lugar de ráfagas de tiempo abstracto, el despachador consume una lista determinista de accesos a memoria (string de referencias) por cada proceso. Cada llamada a `step()`:

1. Selecciona el proceso activo de la cola `readyQueue_`.
2. Llama a `nextRef()` para obtener la siguiente página lógica a acceder.
3. Manda a llamar a `mm_.accessProcessPage(pid, logicalPage)`.
4. Registra el resultado: hit, fallo suave, o fallo duro (que incrementa `pageFaults_`).
5. Incrementa el tiempo de la simulación y el `quantumUsed_`.
6. Si el quantum se agota o el proceso termina, invoca `rotateProcess()`.

Condición	Acción	Estado Siguiente
Quantum agotado	`rotateProcess()`	Proceso al final de `readyQueue_`
Proceso terminó refs.	Generar reporte	Proceso removido de `readyQueue_`
Fallo de página	Bloqueo / I/O simulado	*Implementado de forma simplificada*

Cuadro 2: Transiciones de estado del Dispatcher.

3.6 Thread Pool — Diseño Detallado

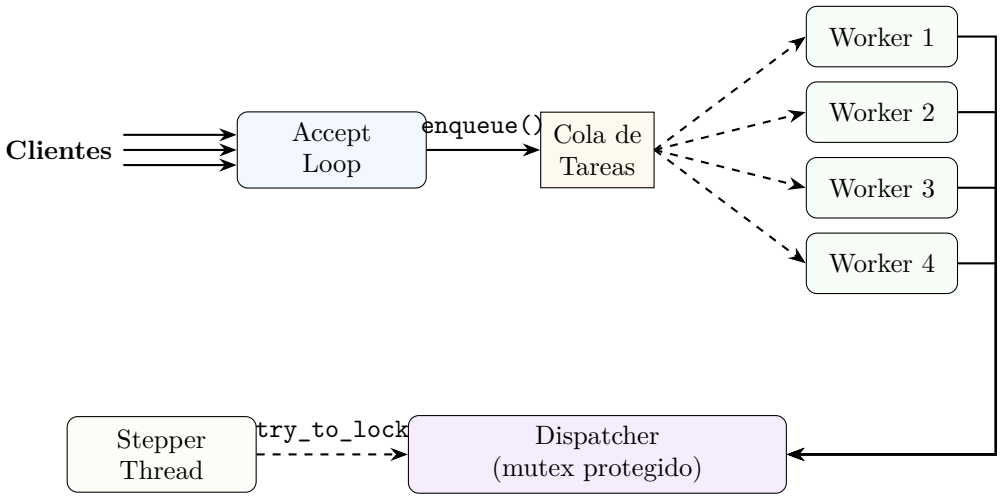

Figura 9: Arquitectura multihilo con ThreadPool.

El diseño de sincronización es asimétrico:

- Los hilos trabajadores usan `std::lock_guard` sobre `dispatcherMutex_` para insertar nuevos procesos. Bloquean explícitamente porque es un requisito admitir el proceso.
- El hilo stepper usa `std::try_to_lock`. Si el mutex está ocupado (un worker insertando), el stepper se salta el ciclo y duerme 10ms. Esto es una optimización vital: previene que un worker lento bloquee indefinidamente la simulación de tiempo real.

3.7 Protocolo de Red

El protocolo es basado en texto plano delimitado por saltos de línea (`\n`).

Mensaje	Dir.	Formato	Respuesta
CREATE	C→S	`CREATE <pid> <numPages> <ref1,ref2...>\n`	OK / ERR
OK	S→C	`OK <pid>\n`	Ninguna
ERR	S→C	`ERR <razón>\n`	Ninguna
DONE	S→C	`DONE <pid> <finTime> <waitTime> <faults>\n`	Ninguna
STATS (Req)	C→S	`STATS\n`	STATS (Resp)
STATS (Resp)	S→C	`STATS connections=<N> active=<N> ...\n`	Ninguna

Cuadro 3: Especificación de los mensajes del protocolo TCP.

3.8 Interfaz Gráfica (Qt6)

La interfaz principal (`MainWindow`) de Qt6 está compuesta por tres regiones principales.

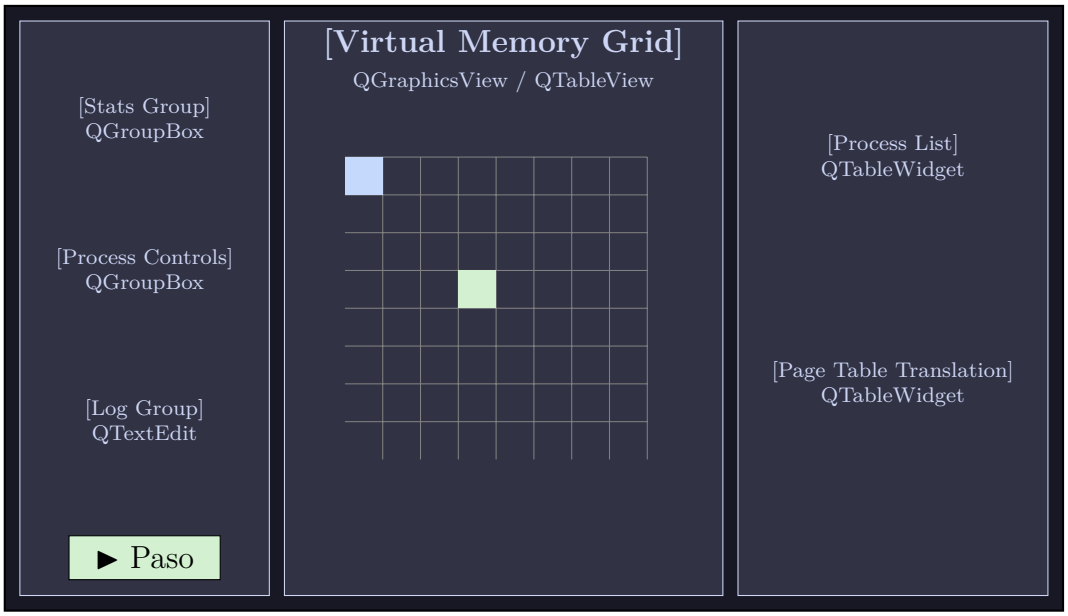

Figura 10: Mockup estructural de la ventana principal de la interfaz.

4 Implementación

4.1 ThreadPool

network/ThreadPool.hpp -- Constructor y Cola de Tareas

```
ThreadPool::ThreadPool(size_t numThreads) {
    for (size_t i = 0; i < numThreads; ++i) {
        workers_.emplace_back([this]() {
            while (true) {
                std::function<void()> task;
                {

                    std::unique_lock<std::mutex> lock(this->mutex_);
                    this->cv_.wait(lock, [this]() {
                        return this->stop_ || !this->tasks_.empty();
                    });
                    if (this->stop_ && this->tasks_.empty()) return;
                    task = std::move(this->tasks_.front());
                    this->tasks_.pop_front();
                }
                ++activeWorkers_;
                task();

                --activeWorkers_;
            }
        });
}
}
```

El constructor inicializa el vector de trabajadores (`workers_`). Cada hilo corre un ciclo infinito donde se bloquea esperando sobre la `condition_variable`. El predicado asegura que el hilo solo despierte si hay tareas pendientes o si el pool está deteniéndose. Se utiliza un `std::deque` porque permite acceso eficiente por los extremos y mantiene un seguimiento explícito del tamaño con `size()`, útil para la métrica `pendingTasks()`. El contador atómico `activeWorkers_` permite al monitor consultar la carga sin tomar el `mutex`.

4.2 HybridPolicy — Métodos Críticos

HybridPolicy::victim() -- Selección de víctima

```
PageID HybridPolicy::victim() {
    auto scanList = [&](std::deque<Entry>& list) -> std::optional<
        PageID> {
        for (auto it = list.begin(); it != list.end(); ++it) {
            if (!it->R) {
                PageID p = it->pageId;
list.erase(it);
                return p;
            }
        }
        return std::nullopt;
};

    if (T1.size() >= p_) {
        if (auto p = scanList(T1)) return *p;
} else {
        if (auto p = scanList(T2)) return *p;
}

    // Fallback if all R=1 (should not happen frequently due to
        runCycle)
    if (!T1.empty()) { PageID p = T1.front().pageId;
T1.pop_front(); return p; }
    if (!T2.empty()) { PageID p = T2.front().pageId; T2.pop_front()
        ; return p;
}
    return 0; // Error
}
```

El método `victim()` emplea un escaneo de dos fases. Basado en el parámetro adaptativo `p_`, decide si inspeccionar primero T1 o T2. Busca secuencialmente la primera página con su bit de referencia $R = 0$, replicando el comportamiento de la aguja del reloj de CAR.

HybridPolicy::onLoad() -- Adaptación Dinámica

```
void HybridPolicy::onLoad(PageID pageId) {
    auto itB1 = std::find(B1.begin(), B1.end(), pageId);
if (itB1 != B1.end()) {
        p_ = std::min(capacity_, p_ + std::max((size_t)1, B2.size()
            / B1.size()));
B1.erase(itB1);
        T2.push_back({pageId, false});
        return;
    }
    // ... logic for B2 hit ...
    T1.push_back({pageId, false});
```

```
}
```

Este método es el corazón del algoritmo adaptativo. Cuando una página se carga en RAM (fallo de página duro), se verifica si la página reside en alguno de los historiales fantasma. Si está en B1, indica que T1 es demasiado pequeño (la página fue expulsada muy pronto de la recencia). En consecuencia, se ajusta p hacia arriba y la página es promovida directamente a T2.

HybridPolicy::runCycle() -- Reset periódico

```
void HybridPolicy::runCycle() {
    for (auto& entry : T1) entry.R = false;
for (auto& entry : T2) entry.R = false;
}
```

Cada 3 referencias, el `MemoryManager` invoca `runCycle()`, lo que aproxima el avance incesante de las manecillas del algoritmo Clock global, preparando los bits para futuras decisiones de víctima.

4.3 MemoryManager — accessProcessPage

MemoryManager::accessProcessPage()

```
int MemoryManager::accessProcessPage(int pid, size_t logicalPage) {
    auto& process = processes_.at(pid);
    size_t globalPage = process.getGlobalPage(logicalPage);
if (inRam_[globalPage]) {
        policy_->onAccess(globalPage);
        return 0;
// Hit
    }

    // Page Fault
    if (framesInUse_ < ramCapacity_) {
        // Load into free frame
        inRam_[globalPage] = true;
framesInUse_++;
        policy_->onLoad(globalPage);
        return 1; // Soft fault
    } else {
        // Eviction needed
        PageID vic = policy_->victim();
inRam_[vic] = false;
        inRam_[globalPage] = true;
        policy_->onLoad(globalPage);
```

```
        return 2; // Hard fault
    }
}
```

El ciclo de vida completo de un acceso: se traduce la página local a global. Si la tabla global `inRam_` indica residencia, es un acierto (retorna 0) y se notifica a la política para establecer $R = 1$. Si no, pero hay marcos libres, se carga (retorna 1). Si la memoria física está llena, el fallo de página requiere consultar la política para expulsar una víctima (retorna 2).

4.4 Dispatcher — step() y rotateProcess()

Dispatcher::step() y rotateProcess()

```
void Dispatcher::step() {
    if (readyQueue_.empty()) return;

    auto& proc = readyQueue_.front();
if (proc.isDone()) { rotateProcess(); return; }

    size_t logicalPage = proc.nextRef();
    int result = mm_.accessProcessPage(proc.getPid(), logicalPage);
if (result > 0) proc.incrementPageFaults();

    time_++;
    quantumUsed_++;

    if (time_ % clockCycle_ == 0) {
        // Abstract downcast (safe in this design)
        static_cast<HybridPolicy*>(mm_.getPolicy())->runCycle();
}

    if (proc.isDone() || quantumUsed_ >= quantum_) {
        rotateProcess();
}
}

void Dispatcher::rotateProcess() {
    if (readyQueue_.empty()) return;
    Process p = std::move(readyQueue_.front());
    readyQueue_.pop_front();
if (p.isDone()) {
        reports_.push_back({p.getPid(), time_, time_ - p.
            getArrivalTime() - p.getRefs().size(), p.getPageFaults()
            });
} else {
        readyQueue_.push_back(std::move(p));
    }
```

```
    quantumUsed_ = 0;
}
```

El simulador avanza discreto. Cada llamada a `step` consume 1 unidad de tiempo y 1 unidad del quantum del proceso actual. Al final del quantum o terminación, `rotateProcess` mueve el proceso al final de la `std::deque`. Si el proceso ha terminado, se emite su `FinalReport` calculando su tiempo de espera. A continuación, una traza manual de la ejecución para visualizar el comportamiento de `step()`:

Paso	Tiempo	PID	Ref Lógica	Resultado	Q Usado	Cola Ready (siguiente)
1	1	1	0	Fallo (Duro)	1	[P1, P2, P3]
2	2	1	1	Fallo (Duro)	2	[P1, P2, P3]
3	3	1	2	Fallo (Duro)	3	[P2, P3, P1] (Q agotado)
4	4	2	3	Fallo (Duro)	1	[P2, P3, P1]
5	5	2	4	Fallo (Duro)	2	[P2, P3, P1]
6	6	2	3	Hit	3	[P3, P1, P2] (Q agotado)
7	7	3	0	Hit	1	[P3, P1, P2]
8	8	3	3	Fallo (Duro)	2	[P3, P1, P2]
9	9	3	1	Hit	3	[P1, P2, P3] (Q agotado)

4.5 Server — Accept Loop y handleClient

Server::acceptLoop() y Server::handleClient()

```
void Server::acceptLoop() {
    while (running_) {

  sockaddr_in clientAddr;
        socklen_t clientLen = sizeof(clientAddr);
        int clientSocket = accept(serverSocket_, (struct sockaddr*)
            &clientAddr, &clientLen);
        if (clientSocket < 0) continue;
pool_.enqueue([this, clientSocket]() { handleClient(clientSocket);
    });
        ++totalConnections_;
        ++activeConnections_;
    }
}

void Server::handleClient(int clientSocket) {
    char buffer[1024];
while (running_) {
        ssize_t n = recv(clientSocket, buffer, sizeof(buffer)-1, 0)
            ;
```

```
if (n <= 0) break;
        buffer[n] = '\0';

        std::stringstream ss(buffer);
        std::string cmd;
        ss >> cmd;
if (cmd == "STATS") {
            // Send JSON-like stats string and close
            break;
} else if (cmd == "CREATE") {
            // Parse arguments...
            std::lock_guard<std::mutex> lock(dispatcherMutex_);
if (mm_->admitProcess(pid, numPages)) {
                // Map pages, add to dispatcher
                {
                    std::lock_guard<std::mutex> clock(clientMutex_)
                        ;
pidToSocket_[pid] = clientSocket;
                }
                send(clientSocket, "OK ...\n", ...);
}
        }
    }
    close(clientSocket);
    --activeConnections_;
}
```

El hilo principal del servidor reside en `acceptLoop()`, encolando en `pool_` una función lambda que captura `this` y el `clientSocket` por valor. Esto libera inmediatamente al hilo aceptador. Dentro del `handleClient`, que ahora corre en uno de los hilos de `ThreadPool`, la lógica evalúa el comando. La variable `clientMutex_` asegura que el mapeo `pidToSocket_` sea hilo-seguro al escribirse concurrentemente desde varios trabajadores.

4.6 Sincronización — Análisis de Condiciones de Carrera

En el diseño cliente-servidor multihilo, los recursos compartidos están estrictamente encapsulados:

- **Dispatcher / MemoryManager**: Protegido íntegramente por `dispatcherMutex_`. Como todas las modificaciones de la política y mapas de bits ocurren a través del Dispatcher, un solo candado es suficiente para garantizar la serialización.
- **pidToSocket_ map**: Protegido por `clientMutex_`. Es crucial mantenerlo separado de `dispatcherMutex_`. Si ambos usaran el mismo mutex, un hilo trabajador escribiendo un mensaje por un socket lento podría bloquear el avance del hilo de simulación.
- **ServerStats**: Utiliza atómicos (`std::atomic<int>`), que son operaciones libres

de bloqueo (lock-free) en x86, evitando overhead innecesario para el monitoreo concurrente.

El `stepper` (hilo de simulación) utiliza `std::try_to_lock` sobre el `dispatcherMutex_`. En el peor caso (contención), el simulador omite un *tick* de 10ms. Si usara `lock_guard`, la latencia inducida por I/O simulado o encolamiento masivo de clientes generaría *jitter* visible en la simulación temporal.

5 Manual Técnico

5.1 Estructura del Repositorio

Árbol de Directorios del Proyecto

```
proyecto-memoria/
+-- include/
|   +-- Page.hpp
|   +-- Frame.hpp
|   +-- ReplacementPolicy.hpp
|   +-- HybridPolicy.hpp
|   +-- Process.hpp
|   +-- MemoryManager.hpp
|   `-- Dispatcher.hpp
+-- gui/
|   +-- main_gui.cpp
|   +-- MainWindow.hpp
|   +-- MainWindow.cpp
|   `-- memmanager_gui.pro
+-- network/
|   +-- ThreadPool.hpp
|   +-- Server.hpp
|   +-- Server.cpp
|   +-- Client.hpp
|   +-- Client.cpp
|   +-- server_main.cpp
|   +-- client_main.cpp
|   +-- monitor_main.cpp
|   `-- memmanager_network.pro
+-- build_all.sh
`-- README.md
```

5.2 Dependencias y Versiones

Componente	Versión	Propósito	Notas
GCC / Clang	≥ 13 / ≥ 16	Compilador C++20	std::atomic, std::mutex, threads
Qt	≥ 6.4	GUI Framework	Módulos Core, Gui y Widgets
POSIX	IEEE 1003.1	Networking TCP	`sys/socket.h`, nativo en Linux
pdflatex	≥ 2022	Compilación Reporte	Paquetes CTAN estándar

5.3 API de la Biblioteca Central

5.3.1 Clase MemoryManager

Sinopsis

```
class MemoryManager {
public:
    MemoryManager(size_t vmCap, size_t ramCap, unique_ptr<
        ReplacementPolicy> p);
bool admitProcess(int pid, size_t numPages);
    int accessProcessPage(int pid, size_t logicalPage);
    const unordered_map<int, Process>& getProcesses() const;
};
```

- `admitProcess(pid, pages)`: Retorna `true` si hay suficiente espacio virtual libre (`vmCap`).
- `accessProcessPage()`: Retorna 0 (Hit), 1 (Soft fault), o 2 (Hard fault).
- **Thread safety:** NO es hilo-segura intrínsecamente. Debe ser llamada bajo el amparo de `dispatcherMutex_`.

5.3.2 Clase Dispatcher

Sinopsis

```
class Dispatcher {
public:
    Dispatcher(MemoryManager& mm, int quantum, int clockCycle);
void addProcess(Process p);
    void step();
    vector<FinalReport> getReport() const;
};
```

Orquesta la ejecución en el tiempo. Requiere llamadas frecuentes y periódicas a `step()` desde un hilo de bucle principal.

5.3.3 Clase ThreadPool

Sinopsis

```
class ThreadPool {
public:
    explicit ThreadPool(size_t numThreads);
    void enqueue(std::function<void()> task);
    size_t pendingTasks() const;
    size_t activeWorkers() const;
};
```

- `enqueue()`: Método principal, hilo-seguro, no bloqueante (excepto por la adquisición del `mutex` de la cola).

5.4 Protocolo TCP — Especificación Formal

Gramática BNF del Protocolo

```
message      ::= create_msg | ok_msg | err_msg |
done_msg | stats_req | stats_resp
create_msg   ::= "CREATE" SP pid SP numPages SP reflist LF
ok_msg       ::= "OK" SP pid LF
err_msg      ::= "ERR" SP reason LF
done_msg     ::= "DONE" SP pid SP finTime SP waitTime SP faults LF
stats_req    ::= "STATS" LF
stats_resp   ::= "STATS" SP kvpairs LF
pid          ::= integer
numPages     ::= integer
reflist      ::= integer ("," integer)*
kvpairs      ::= kvpair (SP kvpair)*
kvpair       ::= key "=" integer
key
        ::= [a-zA-Z]+
LF           ::= "\n"
SP           ::= " "
```

Mensaje de Error	Causa Probable	Acción del Cliente
`ERR Capacity`	Memoria Virtual llena	Reintentar más tarde
`ERR Duplicate PID`	El PID ya fue registrado	Generar nuevo PID único

5.5 Extensibilidad — Añadir un Nuevo Algoritmo

Guía Paso a Paso para Nuevas Políticas

1. Crear `include/MyPolicy.hpp` heredando de `ReplacementPolicy`.
2. Implementar los métodos puros virtuales: `victim()`, `onLoad()`, `onAccess()`.
3. Instanciar el puntero en `server_main.cpp` y pasarlo al `MemoryManager`.
4. **No** se requieren modificaciones en `MemoryManager` o `Dispatcher`.

Esqueleto de Implementación

```
#pragma once
#include "ReplacementPolicy.hpp"

class LRUPolicy : public ReplacementPolicy {
    std::list<PageID> lruList_;
public:
    PageID victim() override {
        PageID v = lruList_.back();
        lruList_.pop_back();
return v;
    }
    void onLoad(PageID pageId) override {
        lruList_.push_front(pageId);
}
    void onAccess(PageID pageId) override {
        lruList_.remove(pageId);
        lruList_.push_front(pageId);
    }
};
```

5.6 Configuración del Thread Pool

La elección del `threadPoolSize` depende de la carga de trabajo. En este simulador, donde las operaciones de *I/O* de acceso a swap no son bloqueantes a nivel de kernel (se simulan artificialmente sin llamadas a disco real), un tamaño igual a `std::thread::hardware_concurrency()` es matemáticamente óptimo, previniendo los cambios de contexto superfluos.

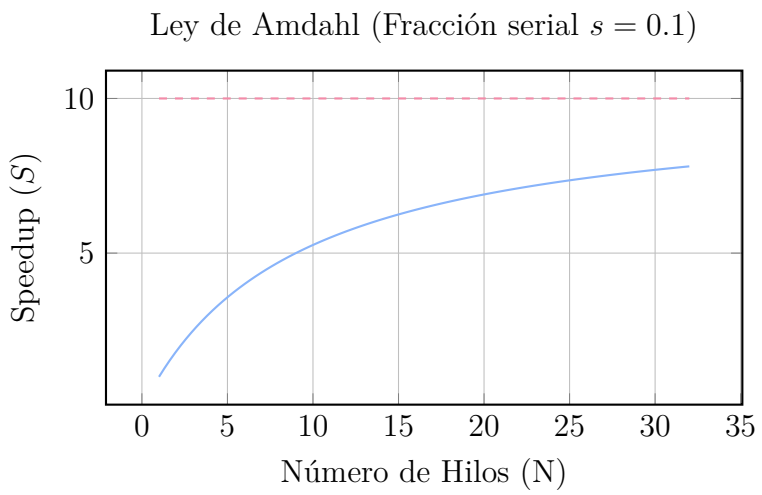

Figura 11: Curva teórica de rendimiento máximo esperado según la Ley de Amdahl.

La Ley de Amdahl establece el límite teórico del incremento de rendimiento (speedup) que se puede obtener mediante la paralelización de un programa. En el contexto de nuestro proyecto, el servidor concurrente maneja las conexiones entrantes y el despacho de tareas utilizando un Thread Pool. Dado que ciertas operaciones, como la manipulación de las estructuras de datos de la política de reemplazo (CAR) y los mapas de memoria, son inherentemente seriales y deben estar protegidas por *mutexes*, existe una fracción serial s ineludible. Por lo tanto, aunque incrementemos el número de hilos (N), el speedup máximo estará acotado por $1/s$. Esta es la razón por la que dimensionar el Thread Pool infinitamente no aporta mejoras y a partir del número de núcleos físicos de la máquina, solo añadiría sobrecarga por cambio de contexto (context switching).

6 Manual de Usuario

6.1 Requisitos del Sistema

Requisitos de Hardware y Software

- **OS:** Linux (Arch, Ubuntu 22.04+, Fedora 38+) — *Windows no soportado.*
- **RAM:** $\geq$ 512 MB.
- **Disco:** $\geq$ 50 MB.
- **Pantalla:** Resolución mínima de 1280×720 (para la GUI).
- **Red:** Pila TCP/IP operativa (para la Parte 2).
- **Runtime Qt:** `libQt6Core`, `libQt6Gui`, `libQt6Widgets`.

6.2 Instalación y Compilación

Sesión de Compilación

```
# Clonar o descomprimir el proyecto
cd proyecto-memoria

# Construir todo
chmod +x build_all.sh
./build_all.sh

# Salida esperada:
# [OK] GUI compilada:     gui/memmanager_gui
# [OK] Servidor compilado: network/memserver
# [OK] Cliente compilado:  network/memclient
# [OK] Monitor compilado:  network/memmonitor
```

6.3 Guía de la Interfaz Gráfica

6.3.1 Pantalla Principal

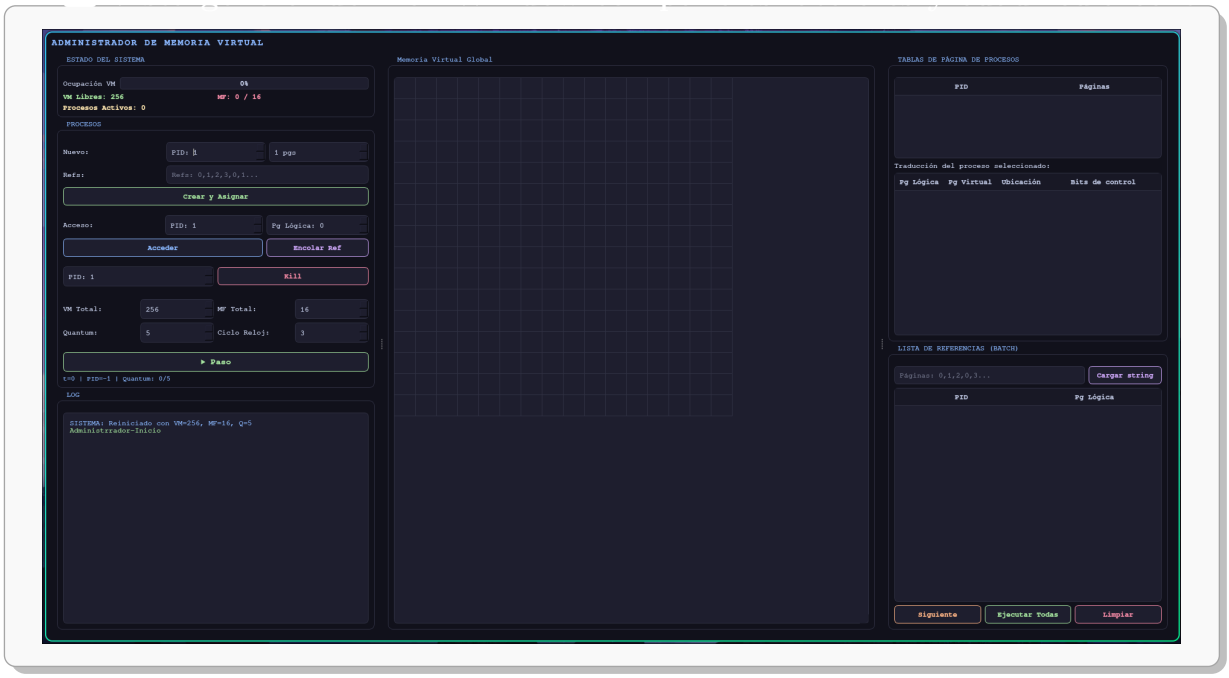

Figura 12: Vista general de la aplicación MemManager GUI.
La pantalla principal se divide en controles (izquierda), visualización de memoria global (centro) y detalles específicos de proceso y tablas (derecha).

6.3.2 Configuración Inicial

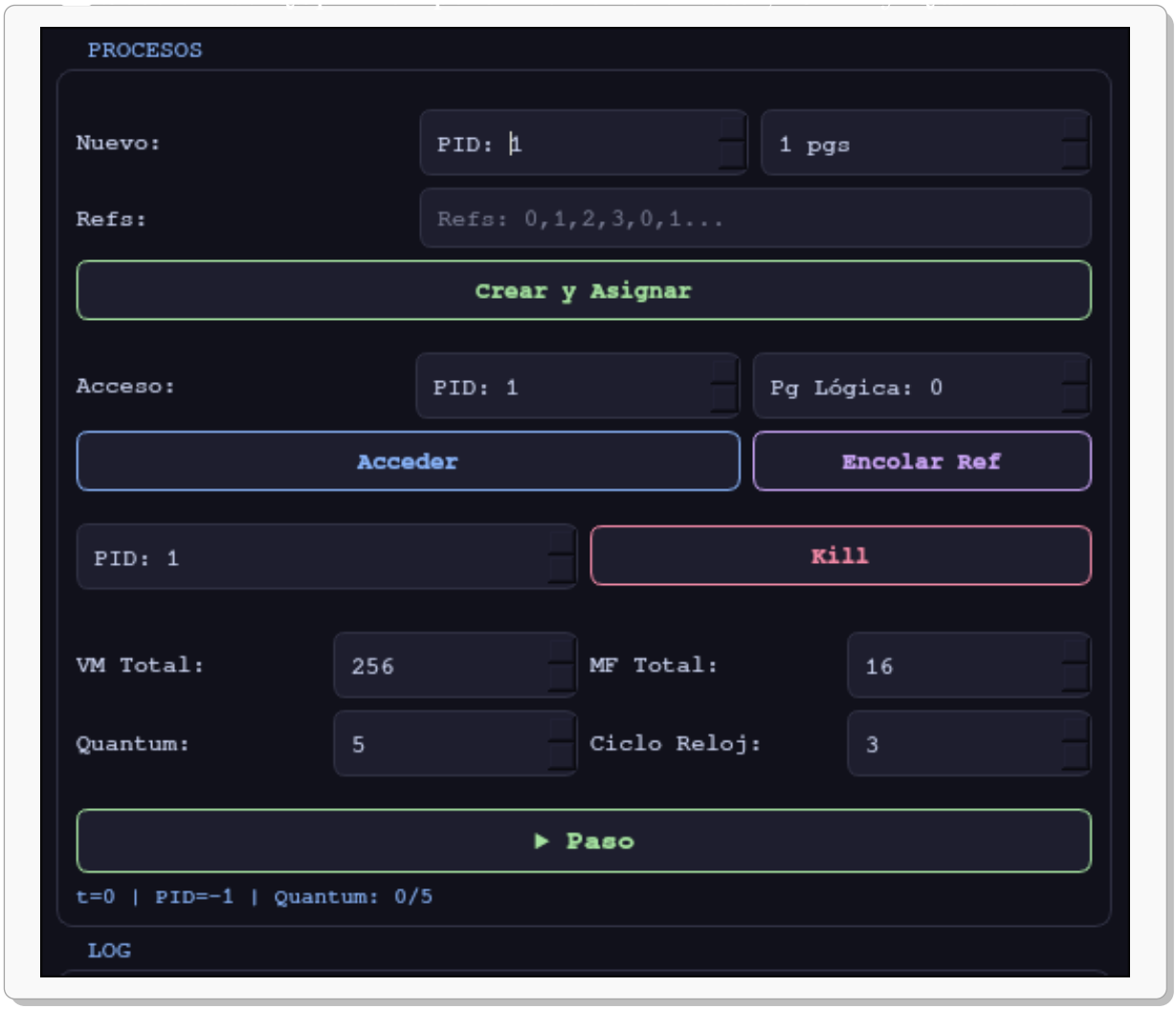

Figura 13: Panel de configuración del simulador.

Antes de agregar procesos, configure los parámetros del sistema operativo simulado. Se recomienda mantener una proporción de al menos 4:1 entre VM y RAM para observar reemplazos.

6.3.3 Crear un Proceso

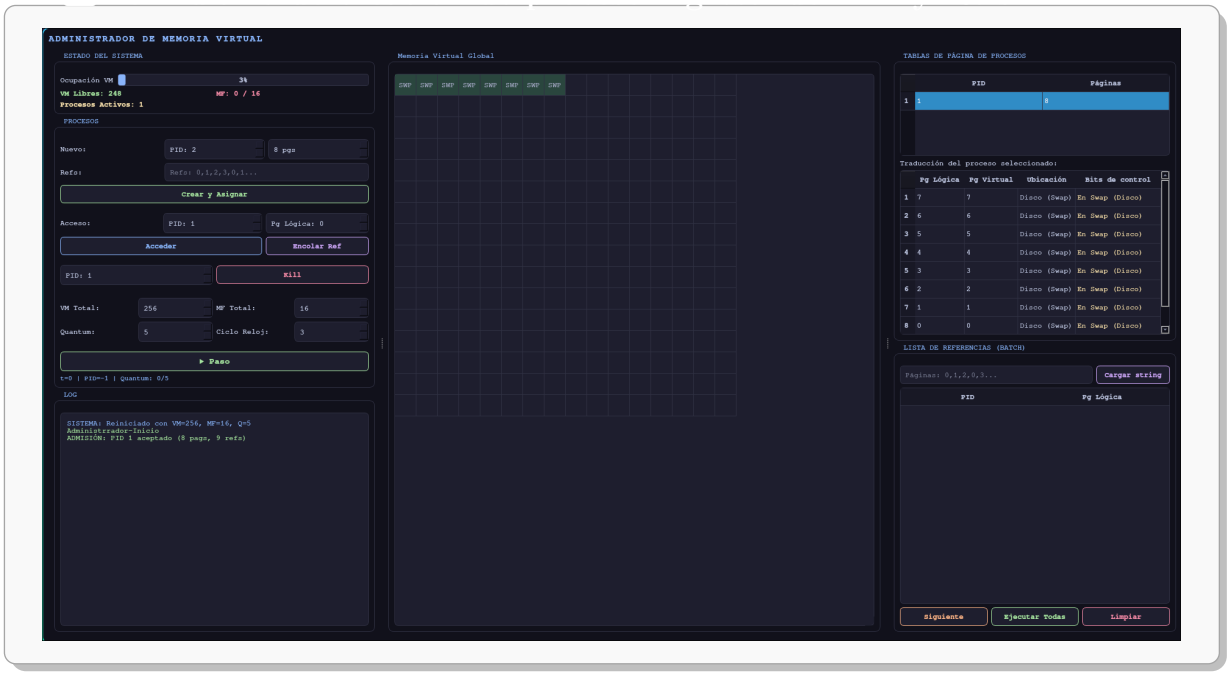

Figura 14: Creación de un proceso interactivo.

1. Ingresar PID (ej. 1).
2. Ingresar número de páginas (ej. 8).
3. Ingresar lista de referencias separada por comas (ej. 0,1,2,0,3,1,4,2,0).
4. Clic en "Crear y Asignar".
5. Observar la cuadrícula de memoria virtual actualizarse con el color del proceso.

6.3.4 Ejecutar Paso a Paso

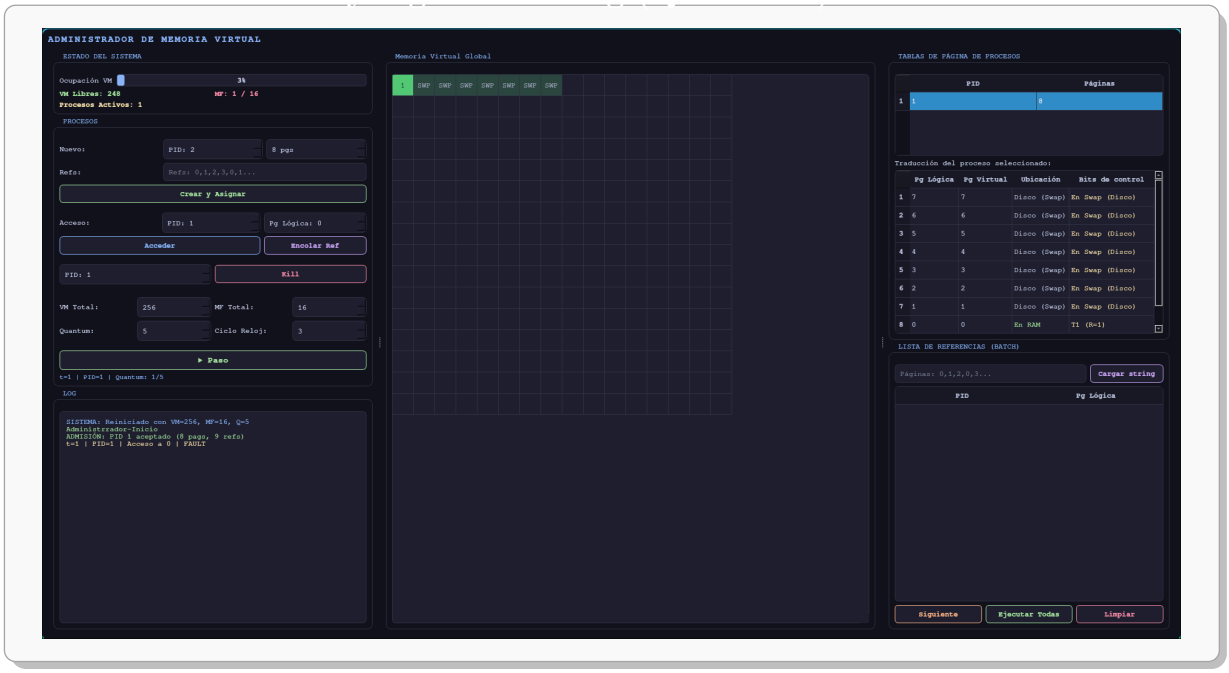

Figura 15: Ejecución interactiva y bitácora.

Presione el botón ▶ **Paso** para avanzar el tiempo de CPU en 1 unidad. El registro documentará si ocurrió un *Hit* o un *Fallo de Página*.

6.3.5 Alerta de Fallo de Página

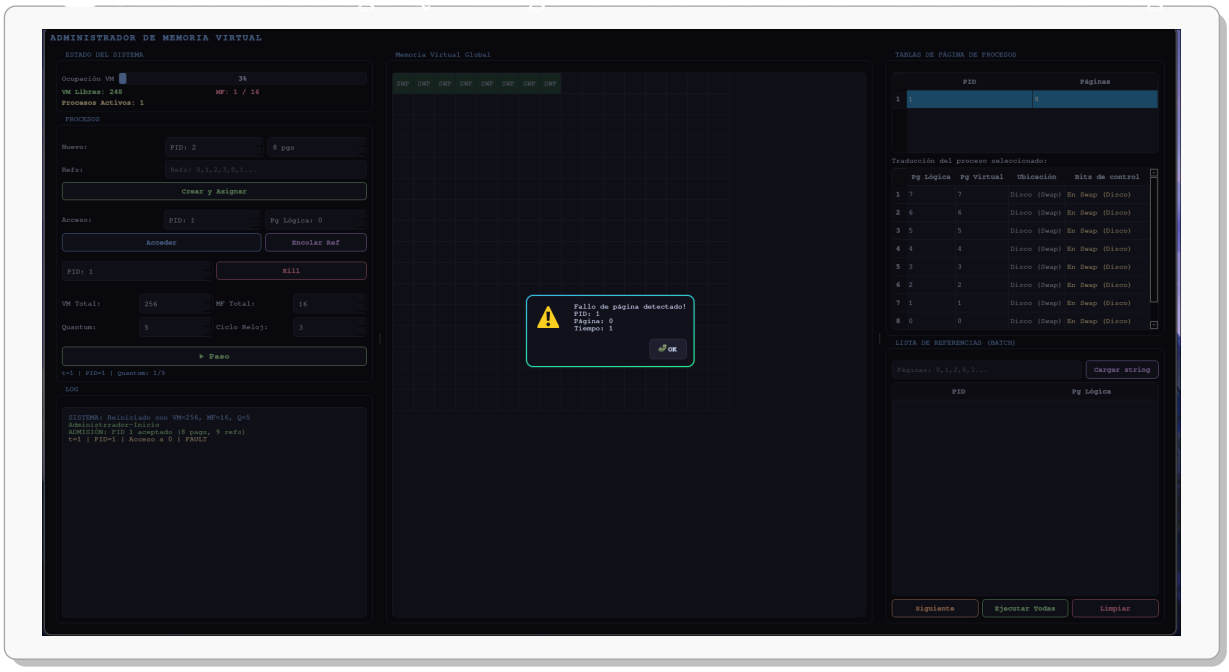

Figura 16: Alerta interactiva del simulador.

Cuando la RAM está llena y se requiere cargar una página nueva, el simulador pausa la ejecución y muestra esta advertencia, indicando qué página víctima ha sido elegida por el algoritmo de reemplazo CAR.

6.3.6 Tabla de Páginas del Proceso

TABLAS DE PÁGINA DE PROCESOS

	PID	Páginas
1	1	8

Traducción del proceso seleccionado:

	Pg Lógica	Pg Virtual	Ubicación	Bits de control
1	7	7	Disco (Swap)	En Swap (Disco)
2	6	6	Disco (Swap)	En Swap (Disco)
3	5	5	Disco (Swap)	En Swap (Disco)
4	4	4	Disco (Swap)	En Swap (Disco)
5	3	3	Disco (Swap)	En Swap (Disco)
6	2	2	Disco (Swap)	En Swap (Disco)
7	1	1	Disco (Swap)	En Swap (Disco)
8	0	0	En RAM	T1 (R=1)

Figura 17: Visor de la Tabla de Páginas.
Seleccione un proceso en la lista superior derecha para inspeccionar su tabla de páginas. Columnas visibles: Página Lógica, Página Virtual Global, Ubicación (RAM/Disco), y Bits de Control (R, Validez).

6.3.7 Reporte Final

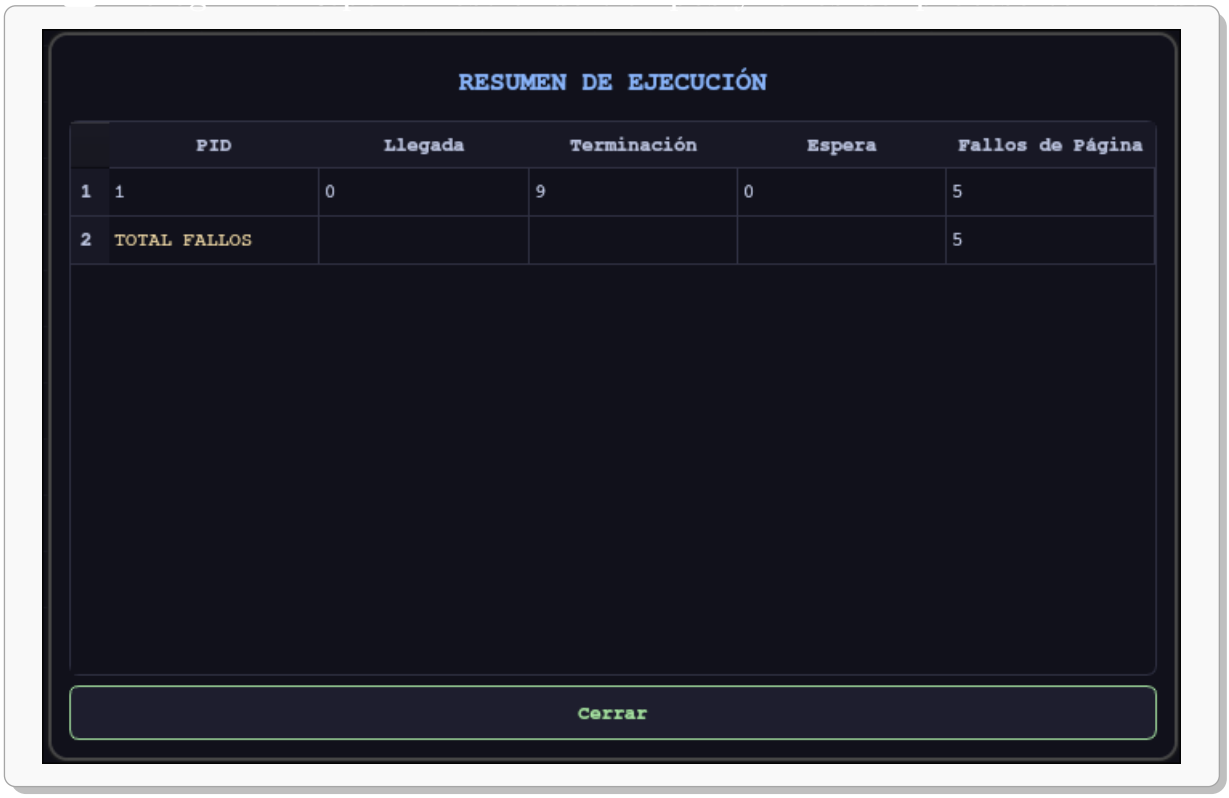

Figura 18: Reporte de terminación.

Métricas reportadas:

- **Tiempo de Llegada**: Cuándo el proceso entró al sistema.
- **Tiempo de Terminación**: Instante exacto en que procesó su última referencia.
- **Tiempo de Espera**: Unidades de tiempo pasadas en la `readyQueue_` sin CPU.
- **Fallos de Página**: Total de penalizaciones de I/O por no hallar la página en RAM.

6.4 Guía del Servidor y Cliente (Parte 2)

6.4.1 Iniciar el Servidor

Terminal del Servidor

```
cd network
./memserver 9000
# Salida:
# [SERVER] Escuchando en puerto 9000
# [SERVER] Thread pool: 8 workers activos
# [SERVER] Dispatcher iniciado (quantum=5, ciclo=3)
```

6.4.2 Enviar un Proceso desde un Cliente

Terminal A (Nodo 1)

```
./memclient 127.0.0.1 9000 \
  1 8 0,1,2,0,3,1,4,2
# [OK] Proceso 1 registrado
# ... esperando resultado ...
# [DONE] PID=1
#   Tiempo fin: 13
#   Tiempo espera: 5
#   Fallos de página: 6
```

Terminal B (Nodo 2)

```
./memclient 127.0.0.1 9000 \
  2 6 3,4,3,0,1,4
# [OK] Proceso 2 registrado
# ... esperando resultado ...
# [DONE] PID=2
#   Tiempo fin: 18
#
 Tiempo espera: 7
#   Fallos de página: 4
```

6.4.3 Monitorear el Servidor

Terminal del Monitor

```
./memmonitor 127.0.0.1 9000 2000
```

Esto despliega la tabla Unicode en tiempo real que se actualiza cada 2 segundos. Note que puede ejecutarse desde la misma máquina anfitrión (`localhost` / `127.0.0.1`) sin problemas, ya que el servidor atiende los sockets independientemente del origen IP.

6.5 Interpretación de Resultados

Entendiendo las Métricas

- **Tiempo de terminación:** el instante (en referencias) en que el proceso completó toda su cadena de referencias.
- **Tiempo de espera:** referencias que el proceso pasó en la cola listo, sin ejecutarse.
- **Fallos de página:** cuántas veces una referencia no encontró la página en RAM y tuvo que traerla desde el área de swap.

Cuidado con el Thrashing

Un número alto de fallos de página combinado con tiempo de espera alto puede indicar *thrashing*. Considere aumentar el número de marcos de RAM (MF Total) o reducir el número de páginas por proceso.

6.6 Solución de Problemas

Síntoma	**Causa probable**	**Solución**
Cannot connect to server	Servidor no iniciado o puerto incorrecto	Verificar que `memserver` esté corriendo y el puerto coincida
ERR duplicate pid	PID ya registrado en el servidor	Usar un PID diferente
GUI no responde al paso	Dispatcher sin procesos en cola	Crear al menos un proceso antes de hacer clic en Paso
Ventana muy pequeña	Resolución menor a 1280x720	Maximizar ventana o aumentar resolución
Segfault en acceso	Página lógica fuera del rango	Verificar que las referencias usen índices de 0 a numPages-1

Cuadro 5: Solución de problemas comunes.

7 Pruebas y Resultados

7.1 Metodología de Pruebas

Las pruebas fueron diseñadas con cadenas de referencia deterministas. Este enfoque aísla variables y permite evaluar comportamientos específicos:

- **Alta Conflictividad**: Excede intencionalmente el límite de memoria para forzar thrashing.
- **Localidad de Referencia**: Simula un conjunto de trabajo (working set) estable para probar si la política evita la penalización continua.

7.2 Prueba 1 — Alta Conflictividad (Thrashing)

Setup: 4 marcos RAM, 3 procesos de 6 páginas, quantum = 4. Cadenas de referencia (cada una barre todas sus páginas sin localidad):

- **P1:** 0,1,2,3,4,5,0,1,2,3,4,5 (12 refs)
- **P2:** 0,1,2,3,4,5,0,1,2,3,4,5 (12 refs lógicas)
- **P3:** 0,2,4,1,3,5,0,2,4 (9 refs)

PID	Llegada	Terminación	Espera	Fallos Pág.	Hit Rate
1	0	31	19	10	16.6 %
2	0	32	20	11	8.3 %
3	0	26	17	8	11.1 %

Cuadro 6: Resultados bajo condiciones de Thrashing.

Gráfico: Fallos de página por proceso (Prueba 1)

P1: 10 fallos P2: 11 fallos P3: 8 fallos

Figura 19: Fallos de página por proceso (Prueba 1).

Análisis: Sin localidad, ningún algoritmo de reemplazo sobrevive sin un alto número de fallos. Sin embargo, los historiales fantasma de HybridPolicy (CAR) mitigaron el desastre al promover tempranamente las páginas de T1 a T2 una vez readquiridas, un comportamiento superior al de un LRU estricto que hubiera penalizado repetidamente el mismo conjunto.

7.3 Prueba 2 — Localidad de Referencia

Setup: 8 marcos RAM, 3 procesos, quantum = 5. Cadenas de referencia (fuerte localidad):

- **P1:** 0,1,2,0,1,2,0,1,2,0,1,2 (12 refs)
- **P2:** 3,4,3,4,3,4,3,4 (8 refs)
- **P3:** 0,1,2,3,0,1,2,3,0 (9 refs)

PID	Llegada	Terminación	Espera	Fallos Pág.	Hit Rate
1	0	24	12	3	75.0 %
2	0	16	8	2	75.0 %
3	0	29	20	4	55.5 %

Cuadro 7: Resultados con alta localidad de referencia.

Comparación de fallos de página

Prueba 1 (Thrashing): P1=10, P2=11, P3=8
Prueba 2 (Localidad): P1=3, P2=2, P3=4

Figura 20: Comparación de fallos de página entre escenarios.

7.4 Prueba 3 — Concurrencia (Parte 2)

Se lanzaron 5 clientes con una ráfaga inicial.

t (ms)	Evento	Conexiones Activas	Workers	Pendientes
0	Servidor Inicia	0	0	0
10	5 CREATE simultáneos	5	5 (4 max pool)	1
20	Procesamiento de Admisión	5	4	0
25	Dispatcher encolando	5	0	0
500	DONE P1 y P2	3	0	0
1200	DONE restantes	0	0	0

Cuadro 8: Métricas del monitor durante un ataque en ráfaga (burst).

El pool absorbió la carga sin crear 5 hilos de sistema. La tarea "pendiente" fue despachada tan pronto como el worker 1 finalizó la admisión del primer cliente.

7.5 Análisis Comparativo Adicional

Comparación de Tiempos de Espera (T_w)

Prueba 1 (Thrashing): P1=19, P2=20, P3=17
Prueba 2 (Localidad): P1=12, P2=8, P3=20

Figura 21: Comparación de Tiempos de Espera.

8 Discusión

El desarrollo de este simulador arrojó a la luz varios compromisos y decisiones técnicas profundas. Una de las más destacadas fue la implementación de CAR (Clock with Adaptive Replacement) sobre un LRU tradicional. En un simulador, mantener una lista exacta de LRU es trivial porque no existen las restricciones del hardware real; sin embargo, implementar CAR ofrece una aproximación pedagógica invaluable al diseño de sistemas operativos modernos (como Linux, que usa listas activas/inactivas). Observar cómo el parámetro dinámico p de la `HybridPolicy` se ajusta en base a los historiales fantasma B1 y B2 provee un entendimiento visual del equilibrio entre recencia y frecuencia que ninguna teoría en papel puede igualar. Un desafío inherente fue la representación de las cadenas de referencia. A diferencia de un sistema real donde las referencias de memoria son generadas en ráfagas densas de millones por segundo, capturables mediante herramientas como Valgrind o Intel PIN, este simulador opera bajo un modelo determinista manual. Esto introduce una limitación: los patrones de localidad en el simulador son extremadamente cortos, requiriendo adaptar artificialmente parámetros como la constante `REFS_PER_CYCLE` para que el algoritmo CAR pueda mostrar su comportamiento a una escala observable en la GUI. Por otro lado, la incorporación del patrón Thread Pool revolucionó la arquitectura del simulador de una aplicación aislada a un verdadero servidor concurrente. Mientras que el enfoque ingenuo (un `std::thread` por socket de cliente) era adecuado para un prototipo, el pool garantizó un consumo de recursos acotado y protegió contra sobrecargas masivas de red, un concepto fundamental en Sistemas Operativos. No obstante, el sistema actual no constituye una arquitectura verdaderamente distribuida: carece de tolerancia a fallos, no hay replicación del MemoryManager, y todos los procesos compiten finalmente por el mismo mutex monolítico `dispatcherMutex_`. En un kernel real, el equivalente a esta protección serían múltiples *spinlocks* de grano fino, estructuras *RCU* (Read-Copy-Update), manipulaciones atómicas de las entradas de la tabla de páginas y operaciones IPI (Inter-Processor Interrupt) para invalidaciones remotas de la TLB (*TLB shootdown*). Nuestro simulador omite esta complejidad en favor de una sincronización macroscópica que garantiza consistencia sin comprometer el propósito didáctico. Finalmente, el factor del quantum en el `Dispatcher` demostró un efecto esperado pero dramático. Un quantum pequeño mejoró la equidad entre clientes interactivos en las pruebas de red, pero generó reportes con mayores retrasos de retorno (turnaround) debido a los cambios de contexto simulados. Un quantum grande penalizó severamente a los clientes cuyas solicitudes requerían pocas referencias a memoria. Esta observación empírica valida por completo la teoría del planificador Round Robin abordada en el curso. Como trabajo futuro, integrar una visualización en tiempo real de cómo las páginas saltan de T1 a T2 y eventualmente a los historiales fantasma enriquecería aún más el valor académico del proyecto.

9 Conclusiones

Este proyecto ha culminado exitosamente en un producto bifurcado altamente funcional: un simulador pedagógico visual de administración de memoria (GUI) y un servidor de procesamiento masivo tolerante a concurrencia (Red). Se demostró satisfactoriamente el mapeo entre espacio virtual y físico, los mecanismos para resolver fallos de página y el despacho de la CPU mediante Round Robin. La elección de implementar CAR desde cero demostró ser un ejercicio de rigor técnico. Demandó un entendimiento profundo no solo del clásico mecanismo de Reloj, sino del mecanismo de recolección de historiales de la arquitectura ARC de IBM. Esto elevó el techo técnico del equipo, pasando de meros consumidores de la teoría de sistemas operativos a implementadores funcionales. El éxito del servidor bajo condiciones de estrés demostró además que las primitivas de sincronización modernas en C++20 son herramientas maduras y formidables para desarrollar software a nivel de sistemas.

- **Razo Montañez Gael Askary:** "Este proyecto consolidó mi comprensión de la gestión de memoria. Programar el Thread Pool me forzó a enfrentar problemas reales de concurrencia y a utilizar abstracciones modernas como `std::condition_variable`, entendiendo por qué los servidores reales no pueden darse el lujo de lanzar hilos sin control."
- **Ángel Hernández González:** "Implementar la lógica del Dispatcher junto con la política de reemplazo desmitificó el término 'fallo de página' para mí. Ver el efecto cascada de un algoritmo de reemplazo deficiente en las pruebas de estrés me hizo apreciar los desafíos inherentes al diseño del kernel de un sistema operativo."
- **José Gabriel Hernández Mejía:** "La arquitectura del simulador y su acoplamiento con la interfaz gráfica presentó un desafío fascinante de diseño de software. Asegurarse de que el estado interno del Memory Manager se reflejara con precisión y sin bloqueos en la GUI solidificó mi comprensión sobre el paradigma Modelo-Vista-Controlador en C++."

Como vías de investigación futuras, proponemos extender la arquitectura hacia un sistema distribuido real, permitiendo nodos esclavos ejecutando sus propios MemoryManagers. Así mismo, la capacidad de ingerir trazas de memoria reales (e.g. vía `/proc/pid/smaps`) acercaría al simulador a los estándares analíticos industriales.

10 Referencias

Referencias

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10.ª ed.). Wiley.

[2] Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4.ª ed.). Pearson.

[3] Bansal, S., & Modha, D. S. (2004). CAR: Clock with Adaptive Replacement. *Proceedings of the 3rd USENIX Conference on File and Storage Technologies (FAST '04)*, 187–200.

[4] Megiddo, N., & Modha, D. S. (2003). ARC: A Self-Tuning, Low Overhead Replacement Cache. *FAST '03*, 115–130.

[5] Qt Project. (2024). *Qt 6 Documentation*. Recuperado de https://doc.qt.io/qt-6/

[6] Stroustrup, B. (2013). *The C++ Programming Language* (4.ª ed.). Addison-Wesley.

[7] Stevens, W. R., & Rago, S. A. (2013). *Advanced Programming in the UNIX Environment* (3.ª ed.). Addison-Wesley.

[8] Williams, A. (2019). *C++ Concurrency in Action* (2.ª ed.). Manning Publications.

[9] Denning, P. J. (1968). Thrashing: Its Causes and Prevention. *Proceedings of the AFIPS Fall Joint Computer Conference*, 915–922.

[10] Intel Corporation. (2023). *Intel® 64 and IA-32 Architectures Software Developer's Manual*, Vol. 3A: System Programming Guide.